

Naive algorithm for Pattern Searching

Last Updated : 20 Apr, 2024



Given **text** string with length **n** and a **pattern** with length **m**, the task is to prints all occurrences of **pattern** in **text**.

Note: You may assume that $n > m$.

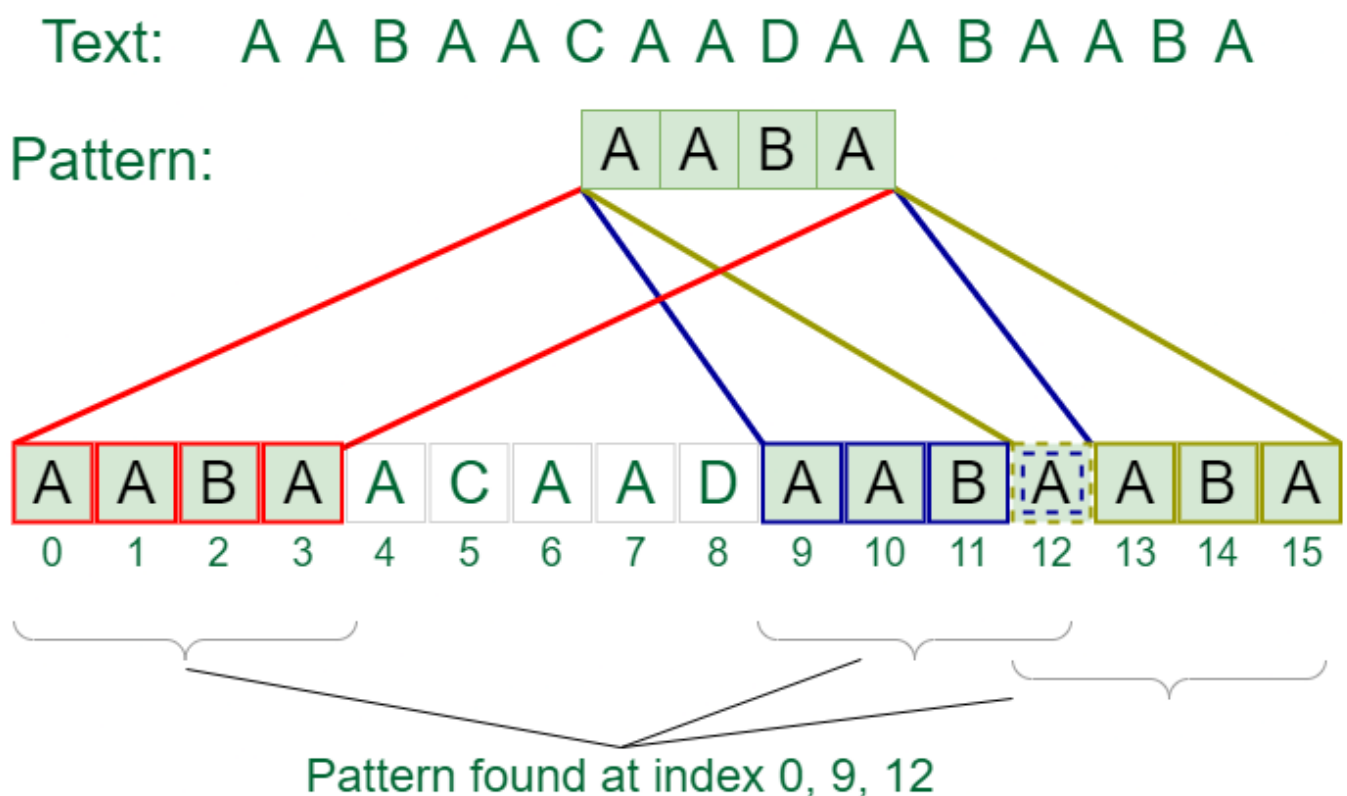
Examples:

Input: text = "THIS IS A TEST TEXT", pattern = "TEST"

Output: Pattern found at index 10

Input: text = "AABAACAADAABAABA", pattern = "AABA"

Output: Pattern found at index 0, Pattern found at index 9, Pattern found at index 12



Pattern searching

Naive Pattern Searching algorithm:

Slide the pattern over text one by one and check for a match. If a match is found, then slide by 1 again to check for subsequent matches.

C++

C

Java

Python3

C#

JavaScript

PHP

```
#include <iostream>
#include <string>
using namespace std;

void search(string& pat, string& txt) {
    int M = pat.size();
    int N = txt.size();

    // A loop to slide pat[] one by one
    for (int i = 0; i <= N - M; i++) {
        int j;

        // For current index i, check for pattern match
        for (j = 0; j < M; j++) {
            if (txt[i + j] != pat[j]) {
                break;
            }
        }

        // If pattern matches at index i
        if (j == M) {
            cout << "Pattern found at index " << i << endl;
        }
    }
}

// Driver's Code
int main() {
    // Example 1
    string txt1 = "AABAACAADAABAABA";
    string pat1 = "AABA";
    cout << "Example 1: " << endl;
    search(pat1, txt1);

    // Example 2
    string txt2 = "agd";
    string pat2 = "g";
```

```
cout << "\nExample 2: " << endl;
search(pat2, txt2);

return 0;
}
```

Output

```
Pattern found at index 0
Pattern found at index 9
Pattern found at index 13
```

Time Complexity: $O(N^2)$

Auxiliary Space: $O(1)$

Complexity Analysis of Naive algorithm for Pattern Searching:

Best Case: $O(n)$.

- When the **pattern** is found at the very beginning of the **text** (or very early on).
- The algorithm will perform a constant number of comparisons, typically on the order of **$O(n)$** comparisons, where n is the length of the **pattern**.

Worst Case: $O(n^2)$.

- When the **pattern** doesn't appear in the **text** at all or appears only at the very end.
- The algorithm will perform **$O((n-m+1)*m)$** comparisons, where n is the length of the **text** and m is the length of the **pattern**.
- In the worst case, for each position in the **text**, the algorithm may need to compare the entire **pattern** against the text.

Naive Algorithm for Pattern Searching

 Comment



kartik

+ Follow

 268



Article Tags :

Strings

Pattern Searching

DSA

Oracle

+3 More

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information